

Render-Fidelity Spike: Math, Code & Tables in A4

chat-to-pdf · GitHub issue #2 · browser phase · self-contained, no network assets

This document is a torture test for the on-device HTML→PDF pipeline. It mixes KaTeX-rendered mathematics, a syntax-highlighted code listing long enough to cross a page boundary, and a wide multi-row table. Everything is inlined so the exact same markup can be handed to `expo-print`'s Android WebView print engine without a server, a build step, or a single network request.

1. Mathematics

Inline math must sit cleanly on the text baseline. The merge-sort recurrence solves to $\mathcal{O}(n \log n)$ time, Gauss's law reads $\nabla \cdot \mathbf{E} = \frac{\rho}{\epsilon_0}$, and the constants $\alpha, \beta, \gamma \in \mathbb{R}$ are ordinary reals embedded mid-sentence. Notice that none of these inline spans should disturb the surrounding line height.

Display equations are centered on their own line. The closed form for a 2×2 inverse:

$$A^{-1} = \frac{1}{\det(A)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

And two classic results — the Gaussian integral and the Basel sum — side by side:

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} \qquad \sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$$

If any glyph above renders as a blank box, a wrong character, or a missing radical/bracket, the embedded font subset is incomplete and the on-device pass will fail the same way.

2. A syntax-highlighted code listing

The following ~90-line Python listing intentionally overflows a single page. A code block taller than one page *must* be permitted to split; `page-break-inside: avoid` should only protect blocks that still fit. Watch the page boundary for a clean break between lines (never mid-glyph) and confirm the monospace font and highlight colors survive.

```

"""A* pathfinding on a weighted grid with a binary-heap open set.

Deliberately long (~90 lines) so the rendered block crosses an A4 page
boundary and we can check that `page-break-inside: avoid` degrades sanely
(a code block taller than one page MUST be allowed to split).
"""

import heapq
import math
from dataclasses import dataclass, field
from typing import Iterator

@dataclass(order=True)
class Node:
    priority: float
    position: tuple = field(compare=False)

class Grid:
    def __init__(self, width: int, height: int, blocked=None):
        self.width = width
        self.height = height
        self.blocked = set(blocked or ())

    def in_bounds(self, pos) -> bool:
        x, y = pos
        return 0 <= x < self.width and 0 <= y < self.height

    def passable(self, pos) -> bool:
        return pos not in self.blocked

    def neighbors(self, pos) -> Iterator[tuple]:
        x, y = pos
        candidates = [
            (x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1),
            (x + 1, y + 1), (x - 1, y - 1), (x + 1, y - 1), (x - 1, y + 1),
        ]
        for c in candidates:
            if self.in_bounds(c) and self.passable(c):
                yield c

    def cost(self, a, b) -> float:
        # diagonal steps cost sqrt(2), orthogonal cost 1
        dx = abs(a[0] - b[0])
        dy = abs(a[1] - b[1])
        return math.sqrt(2) if dx and dy else 1.0

    def heuristic(a, b) -> float:
        """Octile distance: admissible for 8-connected grids."""
        dx = abs(a[0] - b[0])
        dy = abs(a[1] - b[1])
        return (dx + dy) + (math.sqrt(2) - 2) * min(dx, dy)

    def a_star(grid: Grid, start, goal):
        open_set: list[Node] = []
        heapq.heappush(open_set, Node(0.0, start))
        came_from: dict = {start: None}

```

```

g_score: dict = {start: 0.0}

while open_set:
    current = heapq.heappop(open_set).position
    if current == goal:
        return reconstruct(came_from, current)

    for nxt in grid.neighbors(current):
        tentative = g_score[current] + grid.cost(current, nxt)
        if nxt not in g_score or tentative < g_score[nxt]:
            g_score[nxt] = tentative
            priority = tentative + heuristic(nxt, goal)
            heapq.heappush(open_set, Node(priority, nxt))
            came_from[nxt] = current
    return None # no path found

def reconstruct(came_from: dict, current):
    path = [current]
    while came_from[current] is not None:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

if __name__ == "__main__":
    walls = [(2, y) for y in range(0, 8)]
    grid = Grid(10, 10, blocked=walls)
    route = a_star(grid, (0, 0), (9, 9))
    if route is None:
        print("no route")
    else:
        print(f"route of {len(route)} steps: {route}")

```

3. A wide, bordered table

Sixteen data rows across five columns. Borders must stay intact on every cell, the header row should repeat if the table is forced to split, and numeric columns should stay right-aligned.

#	Algorithm	Best	Average	Worst	Stable
1	Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
2	Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes
3	Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No
4	Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
5	Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
6	Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
7	Shell sort	$O(n \log n)$	$O(n^{1.25})$	$O(n^2)$	No
8	Counting sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	Yes
9	Radix sort	$O(nk)$	$O(nk)$	$O(nk)$	Yes
10	Bucket sort	$O(n+k)$	$O(n+k)$	$O(n^2)$	Yes
11	Tim sort	$O(n)$	$O(n \log n)$	$O(n \log n)$	Yes
12	Introsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	No
13	Cocktail sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
14	Gnome sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
15	Comb sort	$O(n \log n)$	$O(n^2/2^p)$	$O(n^2)$	No
16	Cycle sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No

4. Closing prose

If this page renders with correct math glyphs, monospaced and colored code that breaks cleanly across the page boundary, an unbroken bordered table, and no trailing blank page, the browser-side render approach is validated. The remaining risk lives entirely in the delta between Chromium's print engine and Android's WebView print engine, which the on-device phase measures next.

- Fonts are embedded as base64 woff2 — no `@font-face` network fetch.
- Layout uses `display: block / table` only — no flex or grid.
- All page-break hints are expressed with widely-supported CSS 2.1 properties.